In-Depth Source Code Analysis of the multi_brains Embodied Intelligence

In-Depth Source Code Analysis of the multi_brains Embodied Intelligence

1. Course Content
2. Source Code Feature Package Structure Analysis
   - 2.1 Real Feature Package Directory Tree
   - 2.2 Core Module Responsibility Description
3. Large-Model Perception and Decision Hub (model_service.py)
   - 3.1 Dual-Modality Interaction Entry (Voice / Text)
   - 3.2 Dify Large-Model Multimodal Communication and Session Persistence
   - 3.3 Visual Coordinate Alignment and 640×480 Grid Ruler Generation
   - 3.4 BBox Vision Safety Audit Module (bbox_validation.py)
   - 3.5 Action Execution and Status Feedback Closed Loop
4. USB Camera Action Service Source Code Analysis (action_service_usb.py)
   - 4.1 Action Whitelist and Interface Service
   - 4.2 Robotic Arm 2D Visual Grasping Implementation (grasp_obj)
   - 4.3 Bin Placement Action (left_putdown / right_putdown)
   - 4.4 KCF Visual Target Tracking (arm_kcf_track.py)
5. Nuwa Depth Camera Action Service and 3D Spatial Solving (action_service_nuwa.py)
   - 5.1 3D Point-Cloud Back-Projection and Depth Extraction Algorithm
   - 5.2 Robotic Arm Inverse Kinematics (IK) Solving and Grasping (pick)
   - 5.3 Placement at a Designated Spatial Position (GetPlacePoint and place)
   - 5.4 3D Target Stereo Tracking (TargetTrack)
6. Unified System Startup and Hardware Adaptation (llm_agent_control.launch.py)
   - 6.1 USB Mode Startup Chain
   - 6.2 Nuwa Depth Camera Mode Startup Chain
7. Action Preemption, Emergency Stop, and Safety Protection Mechanism
   - 7.1 New-Instruction Preemption and Asynchronous Action Cancellation
   - 7.2 Servo Angle Soft-Limit Protection (clamp_servo_joints)
8. Summary

1. Course Content

In the previous course, we understood the architecture design and action function system of the multi-agent embodied intelligence from a system macro perspective. In this course, we will go deep into the actual code and conduct a comprehensive analysis of the real source code of the car core feature package `multi_brains`.

The `multi_brains` feature package is the core link connecting high-level large-model cognition (Dify / multimodal vision models / voice interaction) with low-level robot motion and robotic arm hardware control (omnidirectional chassis motion, Nav2 navigation, six-DOF robotic arm kinematics solving, spatial grasping and placement).

In the latest system design, `multi_brains` has the following key technical highlights:

- **Dual-hardware architecture adaptation**: natively supports **USB camera mode** (2D visual servo, KCF visual tracking, left/right bin sorting) and **Nuwa depth camera mode** (3D point-cloud spatial distance measurement, TF spatial coordinate back-projection, robotic arm inverse-solving grasping and spatial placement).

- **Multimodal grid alignment protocol**: when taking photos, a precise coordinate grid (640×480) is overlaid on the image, enabling the large model to directly output the target bounding box `[x1, y1, x2, y2]` based on grid pixels, which is then verified by a strict BBox audit module, completely eliminating coordinate drift.
- **ROS 2 Action asynchronous action closed loop**: the brain decision node (`model_service`) and the action execution server (`action_service_*`) use standard ROS 2 Action communication, supporting action execution feedback, status monitoring, and task preemption emergency stop.

> **Tip**
> This course is an in-depth source code analysis course, suitable for advanced users who want to deeply understand the underlying interaction protocols of embodied intelligence, perform secondary development, or extend custom actions. If you only need to experience the robot features, you can skip this course and go directly to the subsequent hands-on chapters.

---

2. Source Code Feature Package Structure Analysis

2.1 Real Feature Package Directory Tree

The `multi_brains` feature package is located at `yahboomcar_ws/src/multi_brains` in the workspace, and its complete directory structure is as follows:

```
multi_brains
├── config
|   ├── map_mapping.yaml          # Mapping table of scene place names to Nav2 absolute world coordinates
|   └── multi_brains_setting.yaml # Core system parameter configuration (Dify address, API key, language, debug switches)
├── language
|   ├── en.yaml                   # English log and voice interaction configuration
|   └── zh.yaml                   # Chinese log and voice interaction configuration
├── launch
|   └── llm_agent_control.launch.py # Unified main launch entry script adaptive to dual-camera environments
├── multi_brains
|   ├── __init__.py
|   ├── action_service_nuwa.py      # Action execution server for Nuwa depth camera mode (3D point-cloud distance measurement, IK inverse-solving grasping and placement)
|   ├── action_service_usb.py       # Action execution server for USB camera mode (2D vision servo, KCF tracking, left/right sorting and placement)
|   ├── asr_detect.py               # Offline voice wake-up and recognition listening node
|   ├── bbox_validation.py          # Large-model output BBox format parsing, coordinate audit, and camera action adapter
|   ├── external_action
|   |   └── arm_kcf_track.py        # Independent node for robotic arm KCF visual target tracking
|   ├── model_service.py            # Large-model decision hub (Dify communication, session management, dual-image upload, Action client)
|   ├── nuwa_coordinate_tracker.py  # Nuwa depth camera 3D point-cloud back-projection and 3D spatial tracking solving engine
```

```
|   ├── tracking_control.py         # Shared vision control utility library (coordinate clamping, distance filtering, state determination)
|   └── utils
|       ├── common_tools.py         # Common utility functions (JSON extraction, multilingual translation logs, process management)
|       ├── dify_client.py          # Dify API HTTP/SSE communication client wrapper
|       └── tts.py                  # Speech synthesis engine (supports offline Piper TTS, Alibaba Tongyi speech, iFlytek speech)
├── package.xml
├── setup.cfg
└── setup.py
```

2.2 Core Module Responsibility Description

1. `config/`:
    - `multi_brains_setting.yaml`: centrally manages the Dify backend address (`DIFY_BASE_URL`), API key (`DIFY_API_KEY`), speech synthesis provider (`TTS_SUPPLIER`), and debug mode switches.
    - `map_mapping.yaml`: defines the conversion relationship from natural-language place names (such as "living room", "sorting area", "charging pile") to ROS 2 map coordinates `(x, y, yaw)`, enabling the large model to understand spatial semantics.
2. `model_service.py`:
    - The robot's "brain cognition and decision hub". It listens for voice wake-up or text input, organizes conversation context to request large-model decisions from Dify, handles multimodal photo taking (generating grid images), parses the action list returned by the large model, and issues the actions one by one to the action server for execution through the ROS 2 Action client.
3. `action_service_usb.py` **and** `action_service_nuwa.py`:
    - The robot's "cerebellum and body action execution end". As ROS 2 Action servers, they receive the brain's action instructions and call the underlying chassis controller, Nav2 navigation stack, robotic arm driver, or vision algorithm library to complete closed-loop actions in the physical world.
4. `nuwa_coordinate_tracker.py`:
    - A spatial 3D solving engine specially designed for the Nuwa depth camera. It completes the back-projection transformation from 2D image pixel coordinates `(u, v)` to 3D points `(X, Y, Z)` in the camera coordinate frame, and converts them to the robotic arm base coordinate frame through the TF tree for robotic arm inverse kinematics (IK) solving.
5. `bbox_validation.py`:
    - Vision safety firewall. The pixel coordinates generated by the large model may suffer from drift, out-of-bounds, or format anomalies. This module strictly verifies the numerical validity of the target bounding box, and automatically adapts the large model's generic action names to the specific hardware functions according to the current camera type (USB / Nuwa).
6. `llm_agent_control.launch.py`:
    - The neural startup hub of the system. It automatically reads the environment variable `CAMERA_TYPE` and dynamically mounts the underlying drivers, TF publishers, kinematics solvers, and action services of the corresponding camera mode.

---

3. Large-Model Perception and Decision Hub (model_service.py)

`model_service.py` is the "brain" of the system, responsible for interfacing upward with the large-model cloud service and commanding the physical hardware downward.

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/model_service.py
```

3.1 Dual-Modality Interaction Entry (Voice / Text)

The `ModelService` node supports two user interaction modes:

1. **Text interaction mode** (`text_chat_mode=True`):
    - Subscribes to the topic `/llm_request_handler` (message type `auto_interfaces/msg/LlmRequest`) to receive text instructions input from the Web end, host computer, or terminal;
    - Decision results are published externally through `/text_response` (`std_msgs/msg/String`), making it convenient for developers to debug or integrate with front-end UI interfaces.
2. **Voice interaction mode** (`text_chat_mode=False`, default mode):
    - `asr_detect.py` listens to microphone audio in a background thread; when a wake word is detected and offline speech-to-text conversion is completed, the recognized text is pushed into the `llm_handler_queue` queue;
    - After the robot makes an action decision or finishes execution, it calls `tts.py` to drive the speech synthesis engine (supporting offline PiperTTS, Alibaba Cloud Tongyi speech, and iFlytek online TTS), and conducts voice Q&A with the user through the onboard speaker.

```python
# Initialize ROS communication and the Action client
def init_ros_comunication(self):
    # Create the Action client connecting to the underlying action server
    self._action_client = ActionClient(self, Rot, "/action_service")
    # Text interaction publisher and subscriber
    self.text_pub = self.create_publisher(String, "text_response", 1)
    self.llm_request_sub = self.create_subscription(
        LlmRequest, "llm_request_handler", self.llm_request_callback, 5
    )
```

3.2 Dify Large-Model Multimodal Communication and Session Persistence

Access to the Dify Agent platform is encapsulated through the `Dify_LLM_Client` class. In order to maintain the contextual coherence of multi-turn conversations, the system maintains `conversation_id` in memory:

- When the user starts a new topic or resets, the `conversation_id` is cleared;
- In multi-turn interactive tasks (such as a sorting task: "first go to the table -> see what is there -> grab

the red building block -> put it on the right"), the same `conversation_id` is carried, ensuring that the large model has complete historical memory.

In multimodal scenarios, `model_service` implements a unique **"dual-channel image upload mechanism"**:

- `raw-picture` **(raw image)**: a clean color frame without any annotations, used for the large model to recognize the object semantics (material, texture, category);
- `grid-picture` **(grid ruler image)**: a frame with a pixel reference grid, used for the large model to accurately estimate the pixel coordinates of the object.

```
def chat(self, input: str, image_path: str = None, raw_image_path: str = None,
         robot_feedback: bool = None, original_query: str = "") -> list[bool, str]:
    kwargs = {
        "inputs": {
            "robot_feedback": str(robot_feedback),
            "original_query": str(original_query),
        },
        "query": input,
        "user": "yahboom",
        "response_mode": "blocking",
        "conversation_id": self.conversation_id
    }

    if image_path is not None:  # Multimodal vision request
        upload_files = []
        # Upload both the grid image with coordinate ticks and the unobstructed raw frame
        for name, path in (("grid-picture", image_path), ("raw-picture", raw_image_path)):
            if path and os.path.exists(path):
                file_id = self.upload_file(path, "image")
                if file_id:
                    upload_files.append({"type": "image", "transfer_method": "local_file",
"upload_file_id": file_id})
        kwargs["files"] = upload_files

    # Send the chat request to Dify
    response = self.client.create_chat_message(**kwargs)
    ...
```

3.3 Visual Coordinate Alignment and 640×480 Grid Ruler Generation

To solve the problem that general multimodal large models cannot accurately perceive the absolute pixel positions of objects in images, the system has a built-in grid ruler drawing function `_draw_coordinate_grid`:

- On the 640×480 resolution frame, 10px is used as the fine-tuning tick, 20px as the regular grid line, and 100px as the highlighted thick line, with specific pixel values annotated (e.g., 100, 200, 300...);
- When the large model views the grid ruler image, it is like consulting a map of longitude and latitude, and can intuitively determine the `[x1, y1, x2, y2]` coordinates of the object's bounding box.

```
def _draw_coordinate_grid(image: np.ndarray) -> np.ndarray:
    annotated = image.copy()
    h, w = annotated.shape[:2]
```

```
    # Draw fine grid lines (20px) and major grid lines (100px with numeric labels)
    for x in range(0, w, 20):
        color = (255, 200, 0) if x % 100 == 0 else (100, 80, 0)
        thickness = 2 if x % 100 == 0 else 1
        cv2.line(annotated, (x, 0), (x, h), color, thickness)
        if x % 100 == 0 and x > 0:
            cv2.putText(annotated, str(x), (x + 2, 20), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255,
255, 255), 1)
    for y in range(0, h, 20):
        color = (255, 200, 0) if y % 100 == 0 else (100, 80, 0)
        thickness = 2 if y % 100 == 0 else 1
        cv2.line(annotated, (0, y), (w, y), color, thickness)
        if y % 100 == 0 and y > 0:
            cv2.putText(annotated, str(y), (5, y - 2), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255,
255, 255), 1)
    return annotated
```

3.4 BBox Vision Safety Audit Module (bbox_validation.py)

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/bbox_validation.py
```

The bounding box data returned by the large model cannot be handed directly to the robotic arm for execution; it must be defensively validated by `bbox_validation.py`:

1. **Size and boundary clamping**: ensure `0 <= x1 < x2 <= 640`, `0 <= y1 < y2 <= 480`, and filter out reversed coordinates (such as `x1 > x2`) or noisy bounding boxes whose width or height is smaller than 10px.
2. **Automatic hardware action adaptation**:
    - If the current running environment is USB camera mode, the generic grasping instruction is uniformly adapted to `grasp_obj(x1, y1, x2, y2)`;
    - If the current mode is Nuwa depth camera mode, it is automatically mapped to the 3D grasping action `pick(x1, y1, x2, y2)`.

```
# Validate the validity of the pixel bounding box
def _pixel_bbox(values: Any) -> list[int] | None:
    if not isinstance(values, (list, tuple)) or len(values) != 4:
        return None
    x1, y1, x2, y2 = [int(v) for v in values]
    if not (0 <= x1 < x2 <= FRAME_WIDTH):
        return None
    if not (0 <= y1 < y2 <= FRAME_HEIGHT):
        return None
    if x2 - x1 < 10 or y2 - y1 < 10:  # Filter out tiny noisy bounding boxes
        return None
    return [x1, y1, x2, y2]
```

3.5 Action Execution and Status Feedback Closed Loop

`model_service` sends the parsed actions to the action server (`action_service_*`) through the ROS 2 Action client, and its core calling mechanism is as follows:

```
# Send the target action to the ROS 2 Action server
def send_action_goal(self, action_string: str):
    goal_msg = Rot.Goal()
    goal_msg.action_name = action_string  # e.g., "pick(210, 180, 290, 260)" or
"navigation(A)"
    self._action_client.wait_for_server()
    send_goal_future = self._action_client.send_goal_async(
        goal_msg, feedback_callback=self.action_feedback_callback
    )
    # Asynchronously wait for the execution result and update the physical feedback status
    ...
```

---

4. USB Camera Action Service Source Code Analysis (action_service_usb.py)

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

In a configuration equipped with a standard USB monocular camera, the system starts the `action_service_usb.py` node. Since the monocular camera lacks depth measurement capability, this service focuses on **2D visual servo alignment, fixed-orientation bin placement, and KCF visual bounding-box tracking**.

4.1 Action Whitelist and Interface Service

This node creates a ROS 2 node named `actioncontroller_node` and provides an `/action_service` ActionServer externally (interface `interfaces.action.Rot`). To ensure the physical safety of the robot, all actions that can be triggered by the large model are registered in the `self.actions` dictionary in a whitelist manner:

```
self.actions = {
    # 1. Moving chassis control
    "set_cmdvel": self.set_cmdvel,
    "move_left": self.move_left,
    "move_right": self.move_right,
    "wait": self.wait,
    "dance": self.dance,
    "cancel": self.cancel,
    "stop": self.stop,
    # 2. Robotic arm pose and emotional interaction
    "arm_up": self.arm_up,
    "arm_down": self.arm_down,
    "arm_shake": self.arm_shake,
```

```
    "arm_nod": self.arm_nod,
    "arm_applaud": self.arm_applaud,
    # 3. Scene cognition and navigation
    "get_current_pose": self.get_current_pose,
    "navigation": self.navigation,
    "seewhat": self.seewhat,
    # 4. 2D vision operations and bin placement
    "grasp_obj": self.grasp_obj,
    "putdown": self.putdown,
    "left_putdown": self.left_putdown,
    "right_putdown": self.right_putdown,
    # 5. Target tracking and line following
    "track": self.track,
    "apriltag_sort": self.apriltag_sort,
    "follow_line_clear": self.follow_line_clear,
    "finishtask": self.finishtask,
}
```

4.2 Robotic Arm 2D Visual Grasping Implementation (grasp_obj)

In USB mode, the robotic arm first moves to a pre-grasp pose so that the camera faces the ground material area. Images are obtained by subscribing to `/usb_cam/image_raw`, and the system works with the persistent grasping node `dofbot_pro_usb`:

1. Receives the pixel bounding box `[x1, y1, x2, y2]` input by the large model;
2. Calculates the pixel offset `(error_x, error_y)` between the center point of the target bounding box and the image center;
3. Adjusts the pan servo of the robotic arm and the chassis speed through a proportional (P) controller, so that the target moves to directly in front of the robotic arm;
4. Starts the servo lowering and gripper closing action to grasp the target object.

4.3 Bin Placement Action (left_putdown / right_putdown)

The car is equipped with fixed storage bins on its left and right sides. In USB mode, after a successful grasp, the system rotates the robotic arm to the left or right bin respectively through preset safe joint angles and releases the gripper:

```
# Preset joint angles for left bin placement (base rotates to the left)
self.putsown_joints = [10, 20, 40, 40, 90, 135]
# Preset joint angles for right bin placement (base rotates to the right)
self.right_putsown_joints = [150, 12, 55, 50, 90, 135]

def left_putdown(self):
    """Precisely place the grasped object into the left bin"""
    self.pubSix_Arm(self.putsown_joints)
    time.sleep(2.0)
    self.pubSingle_Arm(6, 30)  # Open the gripper to release the object
    time.sleep(1.0)
    self.pubSix_Arm(self.init_joints)  # Reset the robotic arm to the driving pose
```

4.4 KCF Visual Target Tracking (arm_kcf_track.py)

When `track(x1, y1, x2, y2)` is executed, the system starts an independent `arm_kcf_track.py` external process:

- The target bounding box given by the large model is used as the initial region of interest (ROI) of the KCF (Kernelized Correlation Filters) tracker;
- The tracker continuously outputs the moving coordinates of the target at a high frame rate of 30fps;
- Combined with the underlying PID controller, the arm servo angles and the chassis movement speed are adjusted in real time to achieve continuous and smooth target following.

---

5. Nuwa Depth Camera Action Service and 3D Spatial Solving (action_service_nuwa.py)

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/nuwa_coordinate_tracker.py
```

The Nuwa depth camera mode is the complete form of the car's embodied intelligence system. Through the HP60C RGB-D sensor, the system simultaneously obtains the 2D color features and 3D spatial depth of objects, supporting truly **spatial 3D grasping, placement at arbitrary spatial points, and 3D stereo tracking**.

5.1 3D Point-Cloud Back-Projection and Depth Extraction Algorithm

`nuwa_coordinate_tracker.py` encapsulates the core of 3D spatial computation. When the large model outputs the target BBox `[x1, y1, x2, y2]` through the grid ruler image:

1. **Foreground depth extraction (`select_foreground_depth_m`)**:
   Extracts the depth image pixel values within the target BBox area. To avoid grasping the floor behind the object or the background tabletop, the algorithm uses median filtering and foreground depth truncation to remove infinite and abnormal depth values.
2. **Pixel back-projection (`pixel_to_camera_point_m`)**:
   Using the intrinsic parameter matrix of the depth camera (focal lengths fx, fy and principal point coordinates cx, cy), converts the center pixel coordinates (u, v) of the target and the depth value Z into the 3D physical coordinate point (Xc, Yc, Zc) in the camera coordinate frame:
   - Xc = (u - cx) * Z / fx
   - Yc = (v - cy) * Z / fy
   - Zc = Z

```
def pixel_to_camera_point_m(u: float, v: float, depth_m: float,
                            fx: float = 554.25, fy: float = 554.25,
                            cx: float = 320.5, cy: float = 240.5) -> tuple[float, float,
float]:
    """Back-project an image pixel point and depth value to 3D physical camera coordinates
(unit: meter)"""
    x = (u - cx) * depth_m / fx
    y = (v - cy) * depth_m / fy
    z = depth_m
    return (x, y, z)
```

3. **TF spatial transformation to the robotic arm base coordinate frame**:
   Using the ROS 2 `tf2_ros` coordinate transform listener, the 3D point `(X_c, Y_c, Z_c)` in the camera coordinate frame is automatically transformed to the robotic arm base coordinate frame `arm_base_link`:

```
# Transform the 3D point from the camera coordinate frame to the robotic arm base coordinate
frame
point_camera = PointStamped()
point_camera.header.frame_id = "camera_link"
point_camera.point.x, point_camera.point.y, point_camera.point.z = X_c, Y_c, Z_c

point_arm_base = self.tf_buffer.transform(point_camera, "arm_base_link",
timeout=rclpy.duration.Duration(seconds=1.0))
```

5.2 Robotic Arm Inverse Kinematics (IK) Solving and Grasping (pick)

After obtaining the actual physical position (X, Y, Z) of the target in the `arm_base_link` coordinate frame, the `kinemarics_dofbot` inverse kinematics solving service is called:

- The target spatial coordinates of the robotic arm end effector and the pose pitch angle are input;
- The inverse kinematics solver calculates the target rotation angles of the 6 servos `[joint1, joint2, joint3, joint4, joint5, joint6]`;
- Smooth trajectory planning is executed, and the robotic arm extends forward, opens the gripper, reaches down, and closes the gripper to complete the stereo grasping.

```
def pick(self, x1: int, y1: int, x2: int, y2: int) -> bool:
    """Precise grasping action solved from Nuwa depth camera 3D spatial coordinates"""
    # 1. Normalize and validate the pixel input
    bbox = normalize_pixel_bbox([x1, y1, x2, y2])
    # 2. Query the depth map and back-projection to get the 3D coordinates (X, Y, Z) of the
object in arm_base_link
    target_3d = self.coordinate_tracker.get_target_3d_position(bbox)
    # 3. Request the robotic arm inverse kinematics solving service
    joint_angles = self.solve_ik(target_3d)
    # 4. Execute the servo grasping action sequence
    return self.execute_arm_trajectory(joint_angles)
```

5.3 Placement at a Designated Spatial Position (GetPlacePoint and place)

Unlike the USB mode, which can only mechanically drop objects into the fixed left and right bins, the Nuwa mode supports the large model specifying an arbitrary placement point through natural language (for example: "put the building block into the green box on the table"):

1. The large model outputs `seewhat()` to observe the scene and identify the green box;
2. The large model specifies the pixel position of the green box and calls `GetPlacePoint(x1, y1, x2, y2)`;
3. The Nuwa depth algorithm solves the spatial physical coordinates (X_place, Y_place, Z_place) of the placement area and saves them in memory;
4. The large model issues the `place()` action, and the robotic arm smoothly moves above these spatial coordinates and precisely lowers the object.

5.4 3D Target Stereo Tracking (TargetTrack)

`TargetTrack(x1, y1, x2, y2)` combines the "gaze" of the robotic arm end camera with the Ackermann/omnidirectional chassis linkage:

- Axes 1 and 2 of the robotic arm adjust pitch and yaw so that the target pixel point always stays stably in the center of the frame ((320, 240));
- The depth algorithm calculates the physical distance Z of the target in real time:
    - If Z > 0.8 m, the chassis follows up and approaches;
    - If Z < 0.4 m, the chassis retreats to maintain a safe distance;
    - implementing stereo visual servo and spatial following.

---

6. Unified System Startup and Hardware Adaptation (llm_agent_control.launch.py)

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/launch/llm_agent_control.launch.py
```

To free users from tedious switching among different scripts, the system provides a unified adaptive startup script for both camera modes.

```
# Read the environment variable CAMERA_TYPE, default is nuwa
camera_type = os.getenv("CAMERA_TYPE", "nuwa").lower()
```

6.1 USB Mode Startup Chain

When `CAMERA_TYPE=usb`, the launch file mounts the following components in order:

1. `usb_cam` node: publishes the `/usb_cam/image_raw` video stream;

2. `laser_bringup` node: starts the LiDAR, providing obstacle avoidance and mapping support;
3. `kinemarics_dofbot` node: provides forward and inverse kinematics solving services for the robotic arm;
4. `dofbot_pro_usb` persistent grasping worker node: handles the 2D visual alignment and grasping state machine;
5. `action_service_usb` node: provides the USB-exclusive action server;
6. `TimerAction` starts `model_service` after a 10-second delay, ensuring that the underlying hardware TF tree is ready.

6.2 Nuwa Depth Camera Mode Startup Chain

When `CAMERA_TYPE=nuwa`, the launch file mounts the corresponding 3D perception stack:

1. `car_base.launch.py`: publishes the entire car base coordinate frames (static and dynamic TF transforms among the chassis, LiDAR, and robotic arm base);
2. `nuwa_vision.launch.py`: starts the Orbbec HP60C RGB-D camera driver and publishes the aligned color image and depth image streams;
3. `grasp_desktop_remove` persistent grasping node: provides the 3D point-cloud grasping action workflow;
4. `action_service_nuwa` node: starts the action server including 3D back-projection and spatial grasping;
5. `model_service` is also started with the same delay.

---

7. Action Preemption, Emergency Stop, and Safety Protection Mechanism

Since embodied intelligent robots operate in the physical world, safety and real-time response are core considerations. `multi_brains` has designed multiple protection mechanisms at both the software and hardware levels:

7.1 New-Instruction Preemption and Asynchronous Action Cancellation

In multi-turn interaction or during an urgent user intervention, if the robot is executing an action that takes a long time (such as during navigation or a long-distance robotic arm grasp) and the user enters a new text or voice instruction:

1. `model_service` determines that the new instruction has higher priority than the currently queued task;
2. The Action client initiates an asynchronous cancel request `cancel_goal_async()` to the server;
3. The action server captures the cancel signal, immediately sends a zero-velocity command to the chassis `/cmd_vel` to brake the vehicle, and allows the robotic arm to brake smoothly.

```
def cancel_active_action(self):
    """Asynchronously preempt and cancel the currently executing action"""
    if self.current_goal_handle is not None and self.current_goal_handle.is_active:
        self.get_logger().warn("[SAFETY] Preemption instruction received, terminating the
current action...")
        self.current_goal_handle.cancel_goal_async()
        self.stop_chassis()  # Immediately publish zero velocity to brake the chassis
```

7.2 Servo Angle Soft-Limit Protection (clamp_servo_joints)

During inverse kinematics calculations or direct large-model control of the robotic arm, extreme angles beyond the physical limits may be solved, causing servo stall burnout or collisions with the vehicle body structure parts.

The system deploys the `clamp_servo_joints` soft-limit filter at the bottom layer of all servo outputs:

- Strictly limits the physical rotation angle range of axes 1 to 6 (for example, axis-1 base limited to `0° ~ 180°`, axis-2 big arm limited to `0° ~ 180°`);
- Any out-of-range angle is forcibly clamped to the safe extreme boundary before the instruction is sent to the bus, ensuring long-term stable operation of the mechanical structure.

---

8. Summary

In this course, we have deeply analyzed all the core implementation details of the `multi_brains` feature package:

- Clarified the design philosophy of the closed-loop division of labor between `model_service` **(brain cognition)** and `action_service_*` **(cerebellum execution)** through standard ROS 2 Action;
- Mastered the essential differences in source code and algorithms between **USB mode** (2D servo and bin sorting) and **Nuwa mode** (3D depth point-cloud back-projection, inverse kinematics spatial grasping);
- Understood the underlying code logic of grid reference ruler alignment, BBox audit firewall, unified adaptive Launch startup script, and the task preemption emergency stop safety mechanism.

After mastering these core principles, in the following chapters we will officially begin the hands-on implementation of concrete embodied intelligence projects!